COMPUTER ENGINEERING
QUEZON CITY

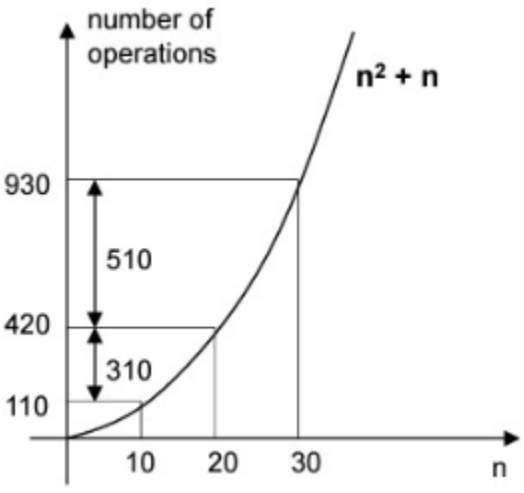
COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	October 7, 2026	**DATE SUBMITTED:**	October 9, 2026
NAME:	Mendoza, Allyson Mendoza, Nathaniel	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

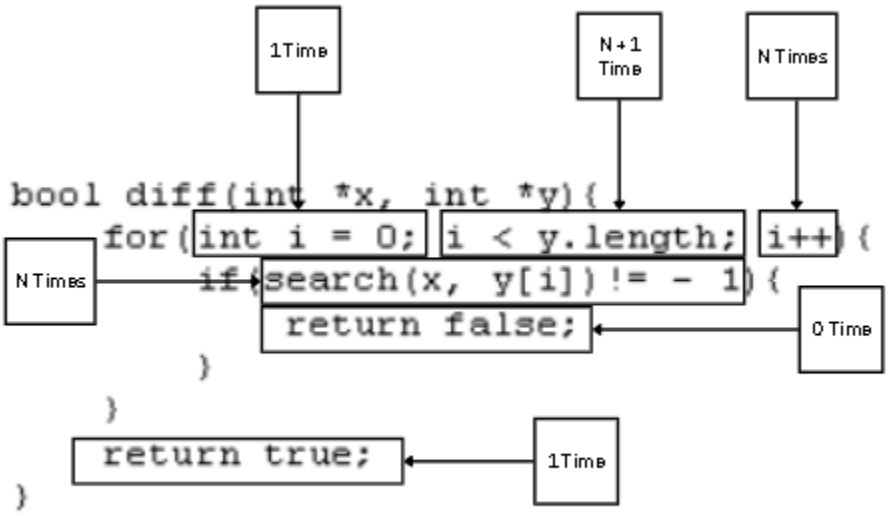
ILO A

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```



n (size of array)	number of operations
10	110
20	420
30	930

ANALYZATION



Operation	Frequency
Initialization of i	1 Time
Comparison of i with y.length	N + 1 Times
Incrementing i	N Times
Calling search(x, y[i])	N Times
search()	N Times
return false	0 Time
return true	1 Time

Since search happens for n times then it will apply n times of m variable comparisons. Therefore

T(m,n) = mn (Wherein each variable is O(N) so n x n = $O(N^2)$

Then worst-case complexity is **$O(N^2)$**

ANALYSIS (MENDOZA, A)

So technically, the worst-case complexity becomes $O(n^2)$ since each operation performs at different times. The first operation runs 1 time only then it will add N times after it will run N times as the search will increment. Then, for the part that will find the search, it will also run for N times as the search may vary. Then, if it is false, it will run 0 times but if it is true, it will run 1 time only.

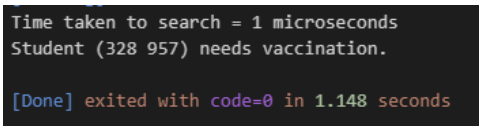
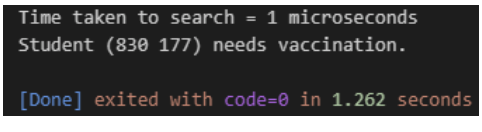
ANALYSIS (MENDOZA, N)

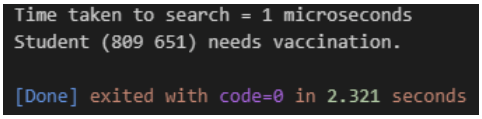
Assume y is being searched in array x. The search runs n times because there are n elements in y. For each search, it can check m elements in array x. Therefore, the total number of comparisons is $n \times m$. Since $m = n$, it becomes $n \times n = n^2$, giving a worst-case complexity of $O(n^2)$.

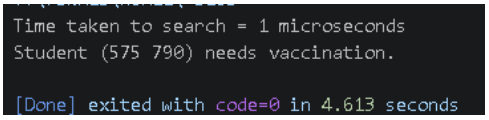
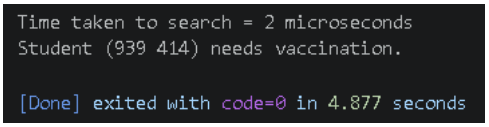
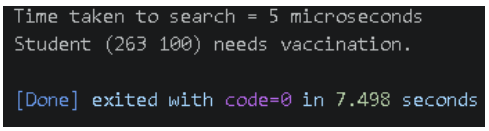
ILO B

Hardware Specifications Comparison		
PC Part	**Mendoza, Allyson**	**Mendoza, Nathaniel**
Processor	Intel(R) Core(TM) i5-8400	Ryzen 5 5500U
RAM	16 GB	16 GB
Storage	954 GB	512 GB
OS	Windows 11	Windows 11

RUNTIME COMPARISONS (*Based off input sizes*)

Mendoza, Allyson			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**
1000	1.148		The execution time was 1.148 seconds as the input size is only 1000.
10000	1.262		The execution time increased since the input size also increased.

100000	2.321		The execution time increased again as the input size became larger compared the previous two input sizes.

Mendoza, Nathaniel			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**
1000	4.613		Searched completed the fastest because the search is small.
10000	4.877		Execution time slightly increased because input size increased to 10,000.
100000	7.498		The execution time almost increased twice compared to the 10,000 input size. Because it went from 10,000 input to a whopping 100,000 size.

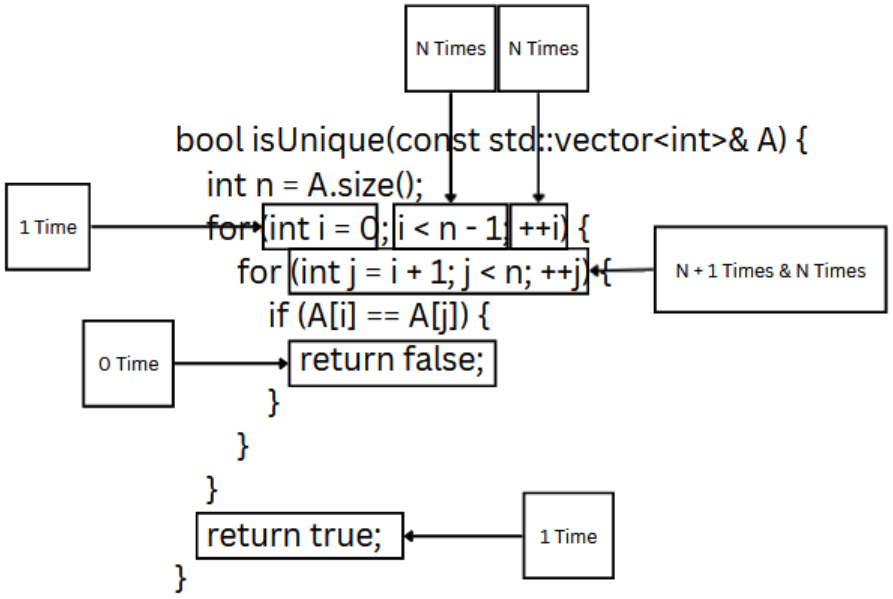
ANALYSIS (MENDOZA, A)
The execution time shows that it will increase as the input size increases. The program took 1.148 seconds for 1000 records then 1.262 seconds for 10000 records and 2.321 seconds for 100000 records. This indicates that the input size can greatly affect its execution time when it runs.
ANALYSIS (MENDOZA, N)
For this part, as I increase the Input size starting from 1,000 to 100,000 the code execution time increased from an additional 200 milliseconds to an additional nearly twice the time taken which is the 7.498 seconds to run the search time via 100,000 input size. Therefore, I can conclude that Hardware Specifications vary on the code runtime, especially on time taken.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1

```
Algorithm Unique(A)
   for i = 0 to n-1 do
     for j = i+1 to n-1 do
       if A[i] equals to A[j] then
         return false
   return true
```

Theoretical Analysis



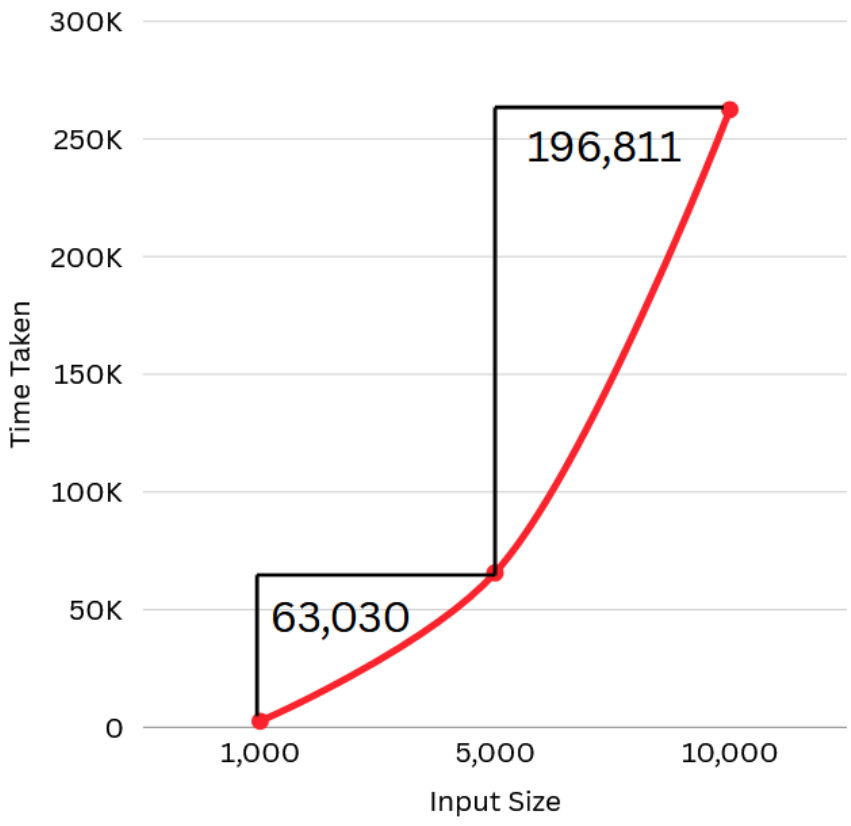
Operation	Frequency
Initialization of i	1 Time
Comparison of i to n - 1	N Times
Incrementation of i	N Times
Inner loop	N + 1 Times & N Times
Return false	0 Time
Return true	1 Time

Worst Case-Complexity is **$O(N^2)$**

Experimental Analysis	
Experimental Analysis Output	
Comparisons	

```cpp
bool isUnique(const std::vector<int>& A) {
    int n = A.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }
    return true;
}
```

```
Input Size (N): 1000 | Time: 2619 us | Result: Unique
Input Size (N): 5000 | Time: 65649 us | Result: Unique
Input Size (N): 10000 | Time: 262460 us | Result: Unique
PS C:\Users\TIPQC\Documents\CPE21S2_MN>
```



Analysis *(Mendoza, A)*
This shows that the time taken is O(n²) since there are 2 loops. Also, it shows that as the input size increases, the time taken will also increase.
Analysis *(Mendoza, N)*
For this part, the time taken increases by O(n^2) as shown in the Experimental Output which shows how big each time it increases by the amount the input size increases.

PROBLEM 2

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3    y = brpower(x, (n-1)/2)
4    return x*y*y
5 if n is even then
6    y = brpower(x, n/2)
7    return y*y
```

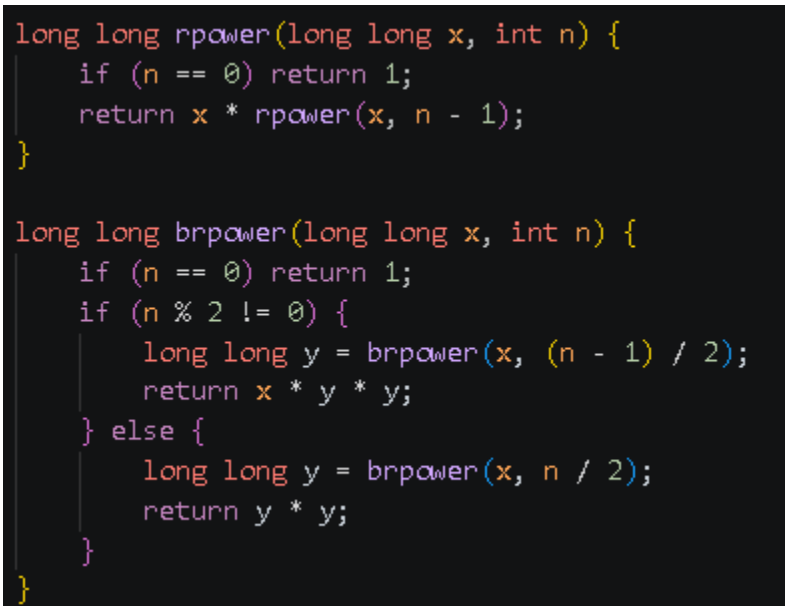
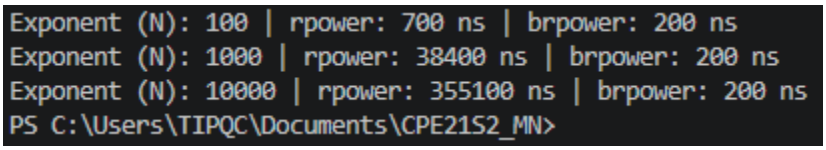
Theoretical Analysis

```
long long rpower(long long x, int n) {
  if (n == 0) return 1;
  return x * rpower(x, n - 1);
}
```

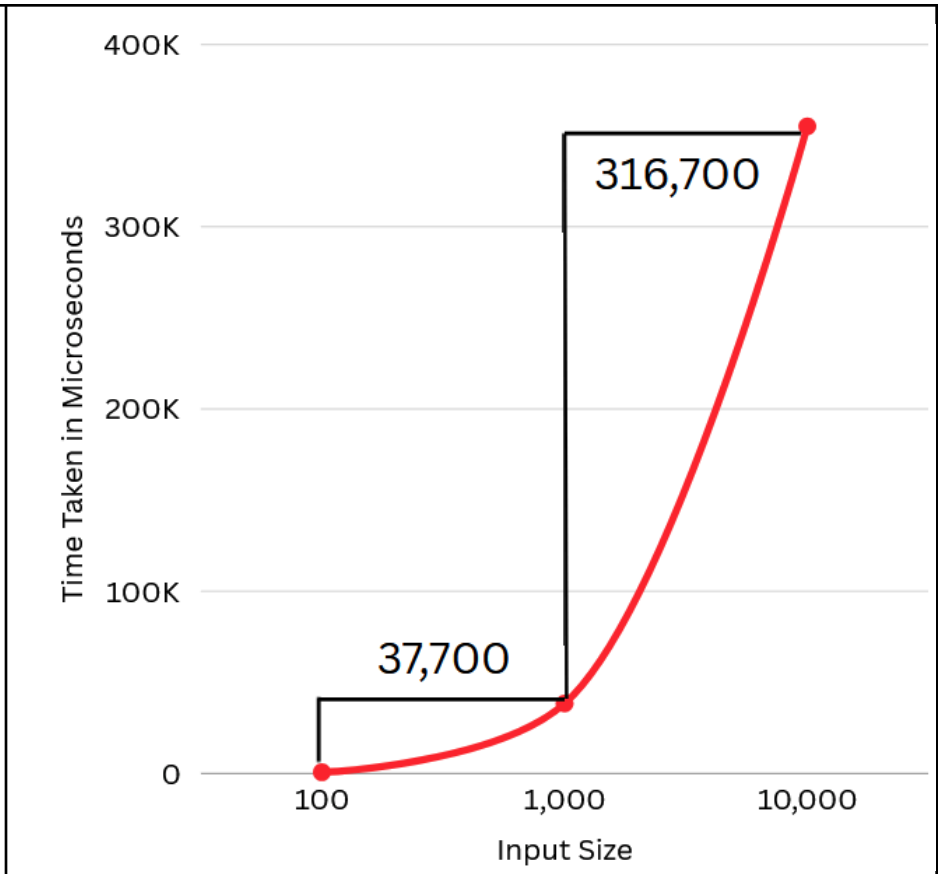
→ O(N)

```
long long brpower(long long x, int n) {
  if (n == 0) return 1;
  if (n % 2 != 0) {
    long long y = brpower(x, (n - 1) / 2);
    return x * y * y;
  } else {
    long long y = brpower(x, n / 2);
    return y * y;
  }
}
```

→ O(log N)

Experimental Analysis	
Experimental Analysis Output	

```
Exponent (N): 100 | rpower: 700 ns | brpower: 200 ns
Exponent (N): 1000 | rpower: 38400 ns | brpower: 200 ns
Exponent (N): 10000 | rpower: 355100 ns | brpower: 200 ns
PS C:\Users\TIPQC\Documents\CPE21S2_MN>
```

Comparisons	

ANALYSIS AND COMPARISON (MENDOZA, A)
The rpower() has a time complexity on O(n) as it decreases the exponent by one in every recursive call while the brpower() has O(log n) as it divides the exponent by two in every recursive call. This shows that the execution time increases as the exponent also increases.
ANALYSIS AND COMPARISON (MENDOZA, N)
This proves that the input increase raises via O(log N) as there wass a very huge gap from 1,000 to 10,000 which therefore shows the exponent to which a fixed number (the base) must be raised to produce the number (n).

CONCLUSION (Mendoza, A)
In conclusion, this hands-on activity shows that theoretical and experimental analysis are very important in determining the algorithm and understanding its flow. The theoretical analysis can predict how an algorithm scales with the input size. While, experimental analysis is the one that measures the overall execution time. Therefore, both of the analysis must be together always and this can help in which algorithm is best suited for a larger inputs. This helped me a lot in identifying and understanding and increasing my knowledge in the concept

of how it works in both theoretical and experimental.

CONCLUSION (Mendoza, N)

After performing, this laboratory activity, I can conclude that each Hardware specification varies on the Code execution runtime wherein it proves and shows to the ILO B where there was almost a big gap when comparing the input size of 100,000 as it shows that the run execution time where there were nearly twice the time taken on both our runtime. Additionally, I improved my analysis in terms of theoretical analysis where I was able to accomplish and analyze the code in a detailed way and probably understand as much as possible to the audience.

REFERENCES

Array containing Duplication - Problem 1
Integer Power Function - Problem 2a
Calculating Power - Problem 2b

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.